An Evaluation of Contracts for C++

Document Number:	D0000R0
Date:	2026-03-11
Audience:	SG21, EWG, All of WG21

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

An Evaluation of Contracts for C++

- Abstract
- Revision History
  - R0: March 2026
- 1. Introduction
- 2. What P2900 Proposes
- 3. The Safety Gap
  - 3.1 Four Definitions
  - 3.2 Terminology as a Political Instrument
  - 3.3 The Institutional Context
  - 3.4 How P2900 Advanced: A Diagnostic
  - 3.5 The Cost
- 4. Design Principles Evaluation
- 5. What Works Well
- 6. What Is Questionable
  - 6.1 Implementation-Defined Semantic Selection
  - 6.2 No `old` Expressions
  - 6.3 Single Global Violation Handler
  - 6.4 Consecutive Evaluation Repetition
  - 6.5 Missing `assume` Semantic
- 7. Gaps Relative to Prior Art
- 8. Coroutine Interaction

9. Standardization Risks

10. Recommendations

Acknowledgements

References

An Evaluation of Contracts for C++

Abstract

This paper evaluates P2900R14[1], "Contracts for C++," targeting C++26. The evaluation begins by asking a question the paper itself does not ask: what is the postcondition of a Contracts facility? It then measures the proposal against that postcondition, identifies gaps relative to prior-art contract systems, analyzes the interaction with C++ coroutines, and examines the political dynamics that shaped the proposal into its current form.

Revision History

R0: March 2026

- Initial revision.

1. Introduction

One starts the evaluation of a language feature by specifying its contract. The first thing to answer is: what is the postcondition?

For a Contracts facility, I think the postcondition is this: a correct program can state its correctness conditions in code, and the language provides a mechanism to check those conditions at runtime with predictable, portable behavior. The checking should not alter the correctness of the program. The cost of checking should be controllable. The conditions should be expressible for the important categories of C++ code: free functions, member functions, class hierarchies, and coroutines.

This paper evaluates P2900R14[1] and its companion rationale P2899R1[2] against this postcondition. Where the proposal satisfies it, the paper says nothing. Where it does not, the paper identifies the gap.

Disclosure. The author has papers before the committee on coroutine-based I/O (P4003[3]) and the interaction between senders and coroutines (P4007[4], P2583[5]). These papers intersect with Contracts at the coroutine interaction described in Section 8. The author has no prior position on Contracts for C++ and no stake in its success or failure. This paper uses AI assistance for research, drafting, and analysis; the author is responsible for all claims.

2. What P2900 Proposes

P2900R14[1] proposes three syntactic constructs for stating contract assertions in code:

```
int f(const int x)
    pre (x != 0)                // precondition specifier
    post (r : r > 0)            // postcondition specifier; r names the return object
{
    contract_assert(x < 100);   // assertion statement
    return x * 2;
}
```

`pre(expr)` is evaluated after parameters are initialized, before the function body. `post(r: expr)` is evaluated after local variables are destroyed on normal return; the optional identifier names the return object. `contract_assert(expr)` is a full keyword evaluated at the point of execution.

Each evaluation of a contract assertion is performed with an evaluation semantic chosen by the implementation:

- `ignore` - no effect; the predicate is still parsed and ODR-uses entities it references
- `observe` - checks the predicate; on violation, calls the violation handler, then continues
- `enforce` - checks the predicate; on violation, calls the violation handler, then terminates
- `quick-enforce` - checks the predicate; on violation, terminates immediately without calling the handler

The mechanism by which the semantic is selected is implementation-defined. The standard recommends `enforce` as the default but does not require it.

When a violation is detected under `observe` or `enforce`, a user-replaceable link-time function receives a `std::contracts::contract_violation` object carrying the source location, predicate text, assertion kind, and the semantic in effect.

3. The Safety Gap

Before evaluating the design, it is necessary to define a term that both proponents and opponents of Contracts use without agreeing on its meaning.

3.1 Four Definitions

"Safety" in current software engineering discourse carries at least four distinct meanings:

- **Behavioral safety.** Fail-fast detection of violated invariants. A program that terminates on a contract violation is "safer" than one that silently continues into corrupted state. This is the sense in which Contracts proponents claim the feature increases safety.
- **Memory safety.** Freedom from buffer overflows, use-after-free errors, and dangling pointer dereferences. This is the sense used in the 2022 NSA Cybersecurity Information Sheet[6], the 2024 White House report on memory-safe languages[7], and the broader government and security industry discourse.
- **Exception safety.** The traditional C++ meaning: no resource leaks under exception propagation, strong or basic guarantees.
- **Formal safety.** Verifiable correctness properties derived from a formal specification; the sense used in the safety-critical and formal methods communities.

When a Contracts proponent says the feature increases safety, the claim is about sense (1). When a critic invokes safety against the proposal, the implied standard is often sense (2). Contracts does not address memory safety. No mechanism in P2900R14[1] prevents a buffer overflow, a use-after-free, or a dangling pointer. Offering behavioral safety as a response to memory-safety concerns is a non-sequitur, whether or not both parties recognize it as one.

3.2 Terminology as a Political Instrument

This confusion is not entirely innocent. "Safety" is politically valuable in the current climate. A proposal framed as "for safety" receives deference that a proposal framed as "for correctness" does not. Opposing something framed as "for safety" requires the opponent to appear to be against safety. The word therefore functions as a political token: attaching it to a position confers rhetorical advantage independent of which definition the speaker intends.

The `observe` semantic illustrates the trap precisely. A program that detects a contract violation and continues executing into potentially corrupted state may be less safe in senses (2) and (4) than a program that does not check at all - because the check creates a false assurance while leaving the underlying defect exploitable. The `enforce` and `quick-enforce` semantics do not have this problem. But the `observe` semantic, presented as a safety feature, is safe only in sense (1) and only if the violation handler terminates the program; when it returns, the "safety" claim depends entirely on what the program does next.

A more precise framing: Contracts for C++ is a correctness tool. When configured with `enforce` or `quick-enforce`, it provides behavioral hardening. When configured with `observe`, it provides diagnostic instrumentation. Neither configuration addresses memory safety, and neither should be evaluated against a memory-safety standard.

3.3 The Institutional Context

The terminological ambiguity did not arise in a vacuum. It operates within an institutional context that rewards it.

Contracts for C++ has been attempted three times before P2900 and failed each time. Phase One (1999-2003), Phase Two (2004-2010), and Phase Three (2016-2020) all produced proposals that were withdrawn or pulled from drafts. P0542[8] reached the C++20 working draft and was pulled after NB ballot controversy over its named build-mode semantics. The pull occurred not because the wording was defective but because the build-mode design gave the room a specific target to object to.

The lesson that shaped P2900: a design that gives opponents a concrete feature to object to will be objected to. A design that removes the concrete feature - by making semantic selection implementation-defined - removes the target. The portability problem is not solved; it is made invisible to the committee at the time of standardization.

A dedicated study group, SG21, was created in 2019 with Contracts as its mandate. A proposal that arrives at EWG bearing "SG21 consensus" carries an imprimatur the room cannot easily interrogate. SG21 consensus means the people who attended SG21 for years agreed, which is not the same as the people who will use the feature agreed.

P2900R14[1] carries 17 co-authors including the GCC maintainer, the libc++ lead, and several figures with institutional standing in WG21. This coalition pre-empts objections from implementers and political gatekeepers before the vote has evaluated the design. Lead author Berne is at Bloomberg, which has a large C++ codebase and documented interest in correctness tooling - a genuine signal of deployment interest.

The "minimum viable product" label creates asymmetric rhetorical pressure. Skeptics who want a smaller feature are told "this is already the minimum." Skeptics who want a larger feature are told "the rest is post-MVP." In November 2024, fourteen revisions into P2900, a poll to remove postconditions (P3487[9]) failed with "consensus against." That this poll was necessary at all reveals that the MVP is a negotiated artifact - the minimum the coalition will accept, not the minimum that delivers user value.

3.4 How P2900 Advanced: A Diagnostic

The following table applies a diagnostic framework for observing how proposals advance through the committee. Each factor is scored against two archetypes: advancement on technical merit (left), and advancement on social consensus (right). A feature that advances primarily through the left column will serve

users well, because it was evaluated against their needs. A feature that advances primarily through the right column will serve its authors' interests, because it was evaluated against their standing.

Factor	Score	Basis
Author's prior work	Mixed	Bloomberg deployment and GCC/Clang implementation are genuine; some co-authors are primarily committee participants
How the room evaluates	Left	P2899R1 records every poll and stated rationale; independent evaluation is supported
Source of objections	Left	Documented objections throughout the record are technical
Response to criticism	Left	14 revisions, each responding to specific documented feedback
Disclosure behavior	Strongly left	P2899R1 is exceptional; deferred features explicitly named with reasons
Complexity handling	Left	Working implementations, examples, companion rationale paper
Scheduling influence	Right	SG21 was created specifically to advance this proposal
Poll composition	Left	Voters' stated reasons reference the design, recorded in the rationale paper
Advancement path	Right	Coalition of 17 institutional co-authors pre-endorses from inside the committee
Patronage role	Right	Co-author list positions institutional gatekeepers as stakeholders before the vote
What blocks the proposal	Left	Blocking attempts - remove postconditions, remove constification - failed on technical grounds
What advances the proposal	Left	Working code at Bloomberg, GCC/Clang implementations, documented SG21 consensus
Credit distribution	Left	Primary authors clearly identified; co-authorship reflects contribution
Failure mode	Mixed	ABI bug caught by implementation (verifiable); impl-defined semantics may be silent post-ship

P2900 scores notably better than most large proposals in recent WG21 history. The disclosure record is exceptional. The implementation experience is genuine. The main right-column pattern is institutional: the dedicated study group, the co-author coalition, and the one design choice - implementation-defined semantics - that is better explained as a political settlement than as a technical solution.

3.5 The Cost

The feature will likely serve users reasonably well because the left-column signals are genuine. The implementation at Bloomberg is real. The working GCC and Clang implementations validate the design.

The political settlement that enabled the proposal, however, transfers the cost of the unresolved portability problem to users. Once the standard ships, whether any given production toolchain checks contracts by default will be determined by vendor decisions made after standardization, with no standard-derived obligation to align. The answer to the most important user question - "will my contracts be checked in production?" - cannot be answered from the standard alone.

The committee would benefit from a shared definition of "safety" before using the term as a criterion in proposal evaluation. A term that can argue both sides of any question is not doing analytical work.

4. Design Principles Evaluation

P2900R14[1] names 16 design principles. Most are sound. A few deserve closer examination.

Prime Directive and Redundancy (Principles 1 and 2). The requirement that contract assertions must not alter the correctness of a correct program, and that removing any subset of assertions from a correct program leaves it correct, is the right foundation. The design is internally consistent with these principles. This is not a small achievement.

Zero Overhead (Principle 4). Sound in intent, but the register-passing ABI interaction - discovered only after a bug in the GCC implementation of caller-side postcondition checking - showed this guarantee is narrower in practice than it appears in principle. The fix required a syntactic restriction that introduces a behavioral asymmetry not apparent from the principle alone.

No ABI Break (Principle 16). Correct and important. The register-passing edge case, however, demonstrated that the boundary between "no ABI break" and "ill-formed program" can be thin.

No Destructive Side Effects and Not Flow Control (Principles 6 and 12). Right in principle. The observable checkpoints introduced via P1494R5[10] partially complicate the claim: an `observe`-semantic assertion that invokes the violation handler and returns is a behavioral change relative to no assertion, even if execution continues. The principle holds for `ignore`; the checking semantics have observable effects by design.

Explicitly Define All New Behavior (Principle 13). Honored throughout the wording. This is a disciplined achievement for a proposal of this size.

The weakest principle in practice is Zero Overhead as applied to semantic selection. The principle guarantees no overhead when not checked, but it cannot guarantee that the default toolchain behavior will check anything at all. Zero overhead on an unchecked assertion is indistinguishable from zero overhead on no assertion.

5. What Works Well

Always-parsed predicates. Unlike `#ifdef NDEBUG`-guarded `assert` macros, a contract predicate under the `ignore` semantic is still parsed and ODR-uses the entities it references. The predicate must always be a well-formed, evaluable expression. This prevents assertion rot - the failure mode where disabled assertions accumulate unreachable code until they cannot be re-enabled without breaking the build.

`contract_assert` as a keyword. A full keyword rather than a macro avoids collision with `<cassert>` and eliminates silent substitution. The `conditional-expression` restriction makes `contract_assert(a = b)` ill-formed without extra parentheses - catching the common typo.

Result-binding syntax. The `post(r: expr)` syntax for naming the return object is clean and extensible. It is a novel feature not available with assertion macros.

Companion rationale paper. P2899R1[2] documents every SG21 poll result, the full history of each design decision, and explicit identification of deferred features with reasons. This level of disclosure is exceptional in WG21.

Separate feature-test macros. `__cpp_contracts` for the language feature and `__cpp_lib_contracts` for the library API, because compiler and library implementations can come from different providers.

6. What Is Questionable

6.1 Implementation-Defined Semantic Selection

The four evaluation semantics are well-designed. The problem is that there is no standardized way to select among them. The mechanism by which semantics are chosen is implementation-defined. The standard recommends `enforce` as the default but does not require it.

In practice, this means a library author cannot write portable contracts with guaranteed runtime behavior. There is no standard mechanism to query which semantic is active or to branch on it. The familiar pattern of checking in debug builds and not checking in release builds cannot be achieved through standard means alone. The most important user-visible question - "will my contracts be checked in production?" - is deferred to toolchain vendors, who have no obligation to align with each other.

This is the political settlement described in Section 3.3: the C++20 predecessor P0542[8] was pulled because named build modes gave the room a target to object to. Implementation-defined selection removes the target. The portability problem is not solved; it is pushed downstream to toolchain configurations that users cannot rely on being consistent.

6.2 No `old` Expressions

Contract predicates treat parameters as implicitly `const`. There is no mechanism to capture the value of a mutable parameter at function entry for use in the postcondition after the function body has executed. Eiffel provides `old` for this purpose; D provides the same.

Without `old`, the class of postconditions that assert mutation - "the output container has the same size as the input had on entry" - cannot be expressed when parameters are passed by mutable reference. This is a real expressiveness gap relative to prior-art contract systems.

6.3 Single Global Violation Handler

One violation handler per program. Libraries cannot install local handlers for their own contract assertions. In a large codebase with distinct ownership boundaries, a library that wants diagnostic output under `observe` semantics must compete for a single global slot with every other library in the program.

6.4 Consecutive Evaluation Repetition

The specification permits a contract assertion to be evaluated more than once per function call. An implementation may repeat any assertion in a consecutive group an implementation-defined number of times. Users migrating from `assert` semantics, which evaluate the predicate exactly once or not at all, will encounter this as a surprising behavioral change. Predicates with side effects that the user expects to run exactly once when checked will behave unpredictably.

6.5 Missing `assume` Semantic

The `assume` semantic - treat the predicate as undefined behavior if false, allowing the optimizer to assume it holds - is listed as a planned future extension. Its absence leaves performance-conscious users without a standard mechanism to communicate contract predicates to the optimizer. The four semantics cover checking but not optimization, making the model feel incomplete.

7. Gaps Relative to Prior Art

Dimension	Eiffel	D	P0542	P2900R14
Preconditions	Yes	Yes	Yes	Yes
Postconditions	Yes	Yes	Yes	Yes
`old` expressions	Yes	Yes	No	No
Class invariants	Yes	Yes	No	No
Virtual function inheritance	Yes	Yes	No	No
Standardized violation API	N/A	N/A	Yes	Yes
Semantic selection model	Fixed	Fixed	Named build modes	Implementation-defined
Coroutine support	N/A	N/A	N/A	Partial
Rationale documentation	N/A	N/A	Absent	Exceptional

P2900R14[1] exceeds prior C++ attempts on documentation quality, the violation handler API, and principled design. It falls behind Eiffel and D on expressiveness: `old` expressions, class invariants, and virtual function inheritance are the features that make those systems genuinely useful for Design by Contract, and all three are absent from or deferred in the MVP.

The deferred features and their cost:

Class invariants. Conditions that must hold for all valid instances of a type, checked on entry and exit of public member functions. This is the primary contribution of Eiffel-style Design by Contract. Without invariants, the MVP cannot express the most important class of correctness condition in object-oriented C++ code.

Virtual function contract inheritance. The Liskov Substitution Principle expressed as a language mechanism: derived-class overrides cannot strengthen preconditions or weaken postconditions. The MVP cannot annotate the most common interface pattern in C++ class hierarchies.

`assume` semantic. Described in Section 6.5.

Assertion labels and levels. The ability to assign symbolic levels (`audit` , `symbolic`) to assertions and map those levels to semantics at build time. Deferred to P3400[11].

The semantic selection model is worse than both the Eiffel and D models (fixed semantics) and the P0542 model (named build modes). Named build modes are at least portable across implementations that adopt the standard. Implementation-defined selection is portable to nothing.

8. Coroutine Interaction

`pre` on coroutines is unproblematic. Preconditions evaluate in the ramp function after parameter initialization, before the coroutine body begins, which is semantically identical to regular functions.

`post` on coroutines is constrained in ways that will surprise users.

Non-reference parameter restriction. Coroutines move parameters into the coroutine frame during the ramp function. By the time the postcondition evaluates - at coroutine completion - those parameters may have been moved-from. If a postcondition ODR-uses a non-reference parameter, the function cannot be defined as a coroutine. This restriction was identified late, after a bug in the GCC implementation of caller-side postcondition checking.

Post-deallocation evaluation. A coroutine's postcondition evaluates when the promise returns normally, which may be after the entire coroutine frame has been deallocated. This is a consequence of how coroutines are specified in C++20, not a rule introduced by Contracts, but it means postconditions on coroutines execute in a context that does not exist for regular functions.

Three-implementation equivalence. P2900R14[1] states the design intent that contracts should behave equivalently whether a function is implemented as a direct coroutine, a coroutine wrapping another coroutine, or a regular function wrapping a coroutine. This goal is not achieved for postconditions that reference parameters: the wrapper pattern requires a non-coroutine outer function, forcing a split between the public API signature and the coroutine implementation.

In practice, postconditions on coroutines are limited to asserting properties of the returned object:

```
awaitable<int> fetch(int id)
    post (r : is_valid(r));
```

Asserting a relationship between input and output when the input is not a reference requires the wrapper pattern. For libraries where coroutines are the primary API surface - networking, async I/O, generators - this is a recurring friction point.

9. Standardization Risks

Ecosystem fragmentation from implementation-defined semantics. Toolchains will select different defaults. Code that works correctly on one toolchain will behave differently on another. Library authors will not be able to document the runtime behavior of their contracts portably. In the absence of any standardized selection mechanism, fragmentation is near-certain. (High likelihood, medium severity.)

NB ballot objections. P0542[8] was pulled during NB ballot review. The mechanism that triggered that pull - controversy over semantic selection - is present in P2900R14[1] in a different form. National bodies that objected to P0542's build modes may object to implementation-defined semantics on the grounds that the portability problem is relocated, not solved. (Medium likelihood, high severity.)

Deferred features requiring core design revision. Class invariants and virtual function contract inheritance have complex interactions with the core design. When these features are designed for C++29, they may reveal that the MVP forecloses certain approaches. The history of C++ features deferring "the hard part" and then discovering that the deferred part requires changes to the already-standardized part is long. (Medium likelihood, medium severity.)

Coroutine restriction surprising users. The non-reference parameter restriction on coroutine postconditions is non-obvious. Severity is low because the wrapper pattern is available, but the frequency of surprise will be high in a post-C++20 codebase. (High likelihood, low severity.)

Late-discovered ABI or interaction issues. The register-passing ABI interaction was discovered via a GCC implementation bug after many revisions. The wording is complex enough that similar interactions may exist. Each discovery after C++26 ships requires a correction paper and potentially an ABI break. (Low likelihood, high severity.)

10. Recommendations

Adopt P2900R14[1] for C++26. The proposal is the strongest Contracts design C++ has produced. The disclosure in P2899R1[2] is exceptional. Working implementations provide evidence of feasibility. The alternative - another decade without Contracts - delivers less value than the MVP with its acknowledged gaps.

Prioritize the `assume` semantic in C++29. It is the highest-value follow-on for performance-conscious users and the least politically contentious, as it adds a new semantic without changing existing ones.

Address `old` expressions before claiming parity with Eiffel and D. Without `old`, the postcondition expressiveness gap relative to prior-art systems is large enough to be a meaningful user complaint.

Publish implementation guidance recommending `enforce` as the universal default. The standard recommends `enforce` but does not require it. A joint vendor statement would substantially reduce the fragmentation risk identified in Section 9.

Design virtual function inheritance before class invariants. Virtual function inheritance has tighter interaction with the existing MVP design. Designing it first reduces the risk that class invariant design later forces changes to already-shipped behavior.

Define "safety" before using it as a criterion. The committee should converge on a shared definition of "safety" before evaluating proposals against it. The current practice - where the word can be deployed for or against any proposal without specifying which definition is intended - does not serve technical evaluation.

Acknowledgements

Thanks to Andrzej Krzemieński for insights on contracts terminology and design methodology that informed the analytical framework of this paper. Thanks to the authors of P2899R1[2] for an exceptional disclosure record that made independent evaluation possible. Thanks to the members of SG21 who spent years building the consensus reflected in P2900R14[1]. AI tools were used for research, drafting, and analysis; the author is responsible for all claims.

References

1. P2900R14 - Joshua Berne, Timur Doumler, Andrzej Krzemieński, et al. "Contracts for C++." February 2025. https://wg21.link/p2900r14

2. P2899R1 - Timur Doumler, Joshua Berne, Andrzej Krzemieński, Rostislav Khlebnikov. "Contracts for C++ - Rationale." February 2025. https://wg21.link/p2899r1

3. P4003 - Vinnie Falco, Mungo Gill. "Coroutines for I/O." February 2026. https://wg21.link/p4003

4. P4007 - Vinnie Falco, Mungo Gill. "Senders and Coroutines." February 2026. https://wg21.link/p4007

5. P2583 - Mungo Gill, Vinnie Falco. "Symmetric Transfer and Sender Composition." March 2026. https://wg21.link/p2583

6. NSA Cybersecurity Information Sheet. "Software Memory Safety." November 2022. https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF

7. White House Office of the National Cyber Director. "Back to the Building Blocks: A Path Toward Secure and Measurable Software." February 2024. https://www.whitehouse.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf

8. P0542 - G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup. ‘‘Support for contract based programming in C++.’’ https://wg21.link/p0542

9. P3487 - Joshua Berne. ‘‘Contracts for C++ - Postcondition Options.’’ https://wg21.link/p3487

10. P1494R5 - Olivier Giroux. ‘‘Partial program correctness.’’ https://wg21.link/p1494r5

11. P3400 - Timur Doumler. ‘‘Contracts: Assertion Levels.’’ https://wg21.link/p3400